

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería

Comandos para un pipeline de datos en Python

Ejercicio 2

Diego Linares

Machine Learning Engineering

Julio 31 de 2026

Plataforma elegida: Python con pandas

Escogí Python porque es la herramienta con la que venimos trabajando en el curso: en el Ejercicio 1 analizamos el dataset de la Champions League con pandas, y en la Actividad 1 construimos el pipeline de preparación de datos con scikit-learn. Usar la misma plataforma deja todo el trabajo amarrado.

Para que los ejemplos no queden en el aire, todos los comandos de este documento están aplicados al mismo archivo que ya conocemos, `champions_league_matches.csv` (151 filas y 18 columnas, de las cuales 7 filas vienen totalmente vacías). Los resultados que aparecen son los que realmente devuelve cada comando con ese archivo.

Las librerías que se usan son:

```
import pandas as pd
import numpy as np
```

1. Extracción de datos

Es el primer paso del pipeline: traer los datos desde donde están hacia el entorno de trabajo. Pandas tiene una familia de funciones `read_*` según el formato de origen.

```
# Desde un archivo CSV (el caso de nuestro dataset)
df = pd.read_csv("champions_league_matches.csv")

# Otros formatos comunes
df = pd.read_excel("archivo.xlsx", sheet_name="Hoja1")
df = pd.read_json("archivo.json")
df = pd.read_parquet("archivo.parquet")

# Desde una base de datos SQL
from sqlalchemy import create_engine
motor = create_engine("postgresql://usuario:clave@servidor:5432/basedatos")
df = pd.read_sql("SELECT * FROM partidos", motor)

# Desde una API que devuelve JSON
import requests
respuesta = requests.get("https://api.ejemplo.com/partidos")
df = pd.json_normalize(respuesta.json())
```

Después de extraer conviene revisar qué fue lo que llegó, antes de seguir:

```
df.shape # (151, 18) -> filas y columnas
df.head() # primeras filas
df.info() # tipos de dato y cantidad de nulos por columna
df.dtypes # solo los tipos
df.describe() # estadísticas de las columnas numericas
```

Para archivos muy grandes que no caben en memoria, se pueden leer por partes:

```
# Lee de 10,000 filas a la vez
for bloque in pd.read_csv("archivo_grande.csv", chunksize=10000):
    procesar(bloque)
```

```
# O traer solo las columnas que interesan
df = pd.read_csv("archivo.csv", usecols=["home_team", "away_team", "result"])
```

2. Filtrado de datos

Filtrar es quedarse solo con las filas y columnas que sirven. Hay varias formas y todas llegan al mismo resultado; la elección depende de qué tan legible quede.

```
# Filtrado por condicion (mascara booleana)
locales = df[df["result"] == "Home Win"] # 71 partidos

# Con query(), que se lee mas natural
locales = df.query("result == 'Home Win'")

# Varias condiciones a la vez: & significa "y", | significa "o"
df[(df["result"] == "Draw") & (df["home_team"] == "Arsenal")]
df[(df["result"] == "Draw") | (df["result"] == "Home Win")]

# isin() para una lista de valores
df[df["result"].isin(["Draw", "Home Win"])] # 96 partidos

# Filtrado por texto
df[df["home_team"].str.contains("Real", na=False)]

# loc para filtrar filas y columnas al mismo tiempo
df.loc[df["result"] == "Away Win", ["home_team", "away_team", "score"]]

# iloc para filtrar por posicion
df.iloc[0:10, 0:5] # primeras 10 filas, primeras 5 columnas
```

También se filtran columnas completas, que es lo que hicimos en la Actividad 1 para quitar las que no aportaban al modelo:

```
df = df.drop(columns=["date", "venue", "referee"])
df = df[["home_team", "away_team", "result"]] # se queda solo con estas
```

Y hay filtros específicos para calidad de datos:

```
df = df.dropna(how="all") # quita las 7 filas totalmente vacias
df = df.drop_duplicates() # quita filas repetidas
```

3. Estandarización de datos

Estandarizar es dejar los datos en un formato consistente. Cubre dos cosas distintas que a veces se confunden: normalizar los tipos y formatos, y llevar los números a una escala común.

Conversión de tipos y formatos

En nuestro dataset varias columnas son números guardados como texto, así que hay que convertirlas:

```
# '63%' -> 63.0
df["home_possession"] = df["home_possession"].str.replace("%", "").astype(float)

# Conversion general a numero; los que no se puedan quedan como nulos
df["columna"] = pd.to_numeric(df["columna"], errors="coerce")

# Texto a fecha
df["date"] = pd.to_datetime(df["date"])

# '3 of 10' -> dos columnas numericas
partes = df["home_shots_on_target"].str.extract(r"(\d+)\s*of\s*(\d+)")
df["tiros_hechos"] = partes[0].astype(float)
df["tiros_intentos"] = partes[1].astype(float)
```

También entra aquí la limpieza de texto, para que no queden variantes del mismo valor:

```
df["home_team"] = df["home_team"].str.strip() # quita espacios sobrantes
df["result"] = df["result"].str.lower() # todo a minusculas
df["home_team"] = df["home_team"].replace({"Man City": "Manchester City"})

# Renombrar columnas a un formato uniforme
df = df.rename(columns={"home_team": "equipo_local"})
df.columns = df.columns.str.lower().str.replace(" ", "_")
```

Estandarización de escala

Cuando las variables tienen rangos muy distintos (la posesión va de 0 a 100 y los tiros de 0 a 20), conviene llevarlas a una escala comparable. Esto es lo que hace `StandardScaler` dentro del pipeline de la Actividad 1:

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Z-score: deja media 0 y desviacion 1
escalador = StandardScaler()
df[["posesion"]] = escalador.fit_transform(df[["posesion"]])

# Min-Max: deja todo entre 0 y 1
df[["posesion"]] = MinMaxScaler().fit_transform(df[["posesion"]])
```

Se puede hacer a mano con pandas, aunque en un pipeline es mejor usar los transformadores de scikit-learn porque aprenden los valores del entrenamiento y los reutilizan:

```
df["z"] = (df["posesion"] - df["posesion"].mean()) / df["posesion"].std()
```

4. Agrupación de datos

Agrupar sirve para resumir: en vez de mirar los 144 partidos uno por uno, se obtienen totales y promedios por categoría. El comando central es `groupby()`.

```
# Cuantos partidos hubo de cada resultado
df.groupby("result").size()

# Promedio de posesion segun el resultado
df.groupby("result")["posesion"].mean()

# Varias metricas a la vez, con nombres propios
resumen = df.groupby("result").agg(
    partidos=("result", "size"),
    posesion_media=("posesion", "mean"),
    tiros_max=("tiros_hechos", "max"),
).round(1)
```

Aplicado a nuestro dataset, ese último comando devuelve:

result	partidos	posesión media
Away Win	48	48.5
Draw	25	48.3
Home Win	71	53.6

Lo interesante es que se ve de una vez que el equipo local gana más seguido y además tiene más posesión en promedio (53.6 % contra 48.5 %), que es justo la ventaja de localía que habíamos notado en el Ejercicio 1.

Otros comandos de agrupación:

```
# Agrupar por varias columnas
df.groupby(["home_team", "result"]).size()

# Tabla dinamica
df.pivot_table(index="home_team", columns="result",
               values="posesion", aggfunc="mean")

# Tabla de frecuencias cruzadas
pd.crosstab(df["home_team"], df["result"])

# transform: devuelve el resultado con el mismo tamaño del original
df["media_del_grupo"] = df.groupby("result")["posesion"].transform("mean")

# Ventanas moviles (por ejemplo, promedio de los ultimos 5 partidos)
df["media_movil"] = df["posesion"].rolling(window=5).mean()
```

5. Manejo de nulos o valores faltantes

Antes de decidir qué hacer con los nulos, hay que ver cuántos hay y dónde están.

```
df.isna().sum() # nulos por columna
```

```
df.isna().sum().sum() # total de nulos (129 en nuestro CSV crudo)
df.isna().mean() * 100 # porcentaje de nulos por columna
df[df["home_saves_pct"].isna()] # ver las filas con nulos
```

A partir de ahí hay dos caminos: eliminar o rellenar.

Eliminar

```
df.dropna(how="all") # solo si TODA la fila esta vacia
df.dropna(subset=["result"]) # si falta la variable objetivo
df.dropna(thresh=10) # deja las filas con al menos 10 datos
df.dropna(axis=1) # elimina columnas con nulos
```

Rellenar (imputar)

```
# Con un valor fijo
df["columna"] = df["columna"].fillna(0)

# Con una estadística de la propia columna
df["home_saves_pct"] = df["home_saves_pct"].fillna(df["home_saves_pct"].median())
df["posesion"] = df["posesion"].fillna(df["posesion"].mean())

# Con el valor mas frecuente (util en categoricas)
df["metodo"] = df["metodo"].fillna(df["metodo"].mode()[0])

# Arrastrando el valor anterior o el siguiente (util en series de tiempo)
df["columna"] = df["columna"].ffill()
df["columna"] = df["columna"].bfill()

# Interpolando
df["columna"] = df["columna"].interpolate()
```

Dentro de un pipeline conviene usar los imputadores de scikit-learn, porque aprenden el valor con los datos de entrenamiento y aplican exactamente ese mismo valor a los datos nuevos, lo que evita fuga de información:

```
from sklearn.impute import SimpleImputer, KNNImputer

imputador = SimpleImputer(strategy="median") # o "mean", "most_frequent", "constant"
X = imputador.fit_transform(X)

# Estima el faltante mirando los registros mas parecidos
X = KNNImputer(n_neighbors=5).fit_transform(X)
```

En nuestro dataset los nulos de `home_saves_pct` vienen de un caso concreto: cuando el portero tuvo “0 of 0” atajadas, el porcentaje es una división entre cero y queda indefinido.

6. Publicar o disponibilizar los datos

Es el último paso del pipeline: dejar los datos ya limpios donde otros puedan consumirlos.

Guardar en archivos

```
df.to_csv("partidos_limpios.csv", index=False, encoding="utf-8")
df.to_excel("partidos_limpios.xlsx", index=False)
df.to_json("partidos_limpios.json", orient="records")

# Parquet: mas liviano y rapido, el formato tipico en data lakes
df.to_parquet("partidos_limpios.parquet", compression="snappy")
```

Escribir en una base de datos

```
from sqlalchemy import create_engine

motor = create_engine("postgresql://usuario:clave@servidor:5432/basedatos")
df.to_sql("partidos_limpios", motor, if_exists="replace", index=False)
```

Exponer los datos por una API

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/partidos")
def obtener_partidos():
    return df.to_dict(orient="records")
```

Compartir el proceso completo como paquete

Otra forma de disponibilizar datos, y la que usamos en la Actividad 1, es empaquetar el código junto con el dataset para que cualquier compañero lo instale y obtenga exactamente los mismos datos:

```
# pip install .
from act1_pipeline import extraer_datos, filtrar_datos
df = filtrar_datos(extraer_datos())
```

Guardar el modelo o el pipeline ya entrenado

```
import joblib

joblib.dump(pipe, "pipeline.pkl") # guardar
pipe = joblib.load("pipeline.pkl") # cargar en otra maquina
```

Cómo se conecta todo en un pipeline

Siguiendo la estructura que plantea el artículo de DataCamp, los seis puntos anteriores se ordenan así dentro de un pipeline de datos:

Etapas del pipeline	Punto	Comandos principales
Extract	1	<code>read_csv</code> , <code>read_sql</code> , <code>read_json</code>
Transform	2	<code>query</code> , <code>loc</code> , <code>isin</code> , <code>drop</code>
Transform	3	<code>astype</code> , <code>to_datetime</code> , <code>StandardScaler</code>
Transform	5	<code>isna</code> , <code>fillna</code> , <code>SimpleImputer</code>
Transform	4	<code>groupby</code> , <code>agg</code> , <code>pivot_table</code>
Load	6	<code>to_csv</code> , <code>to_parquet</code> , <code>to_sql</code>

El artículo distingue entre ETL y ELT según en qué momento se transforman los datos: en ETL se transforma antes de cargar, y en ELT se carga primero en crudo y se transforma después, dentro del mismo almacén. Con pandas normalmente se trabaja al estilo ETL, porque las transformaciones se hacen en memoria antes de guardar el resultado.

Una recomendación del artículo que vale la pena rescatar es diseñar el pipeline solo tan complejo como haga falta. Para un dataset de 151 filas como el nuestro, pandas y scikit-learn son suficientes; herramientas como Spark o Airflow tienen sentido cuando el volumen o la frecuencia de actualización lo justifican.

Referencias

- DataCamp. Introduction to Data Pipelines for Data Professionals. <https://www.datacamp.com/tutorial/introduction-to-data-pipelines-for-data-professionals>
- Pandas. User Guide. https://pandas.pydata.org/docs/user_guide/index.html
- scikit-learn. Preprocessing data. <https://scikit-learn.org/stable/modules/preprocessing.html>
- scikit-learn. Imputation of missing values. <https://scikit-learn.org/stable/modules/impute.html>